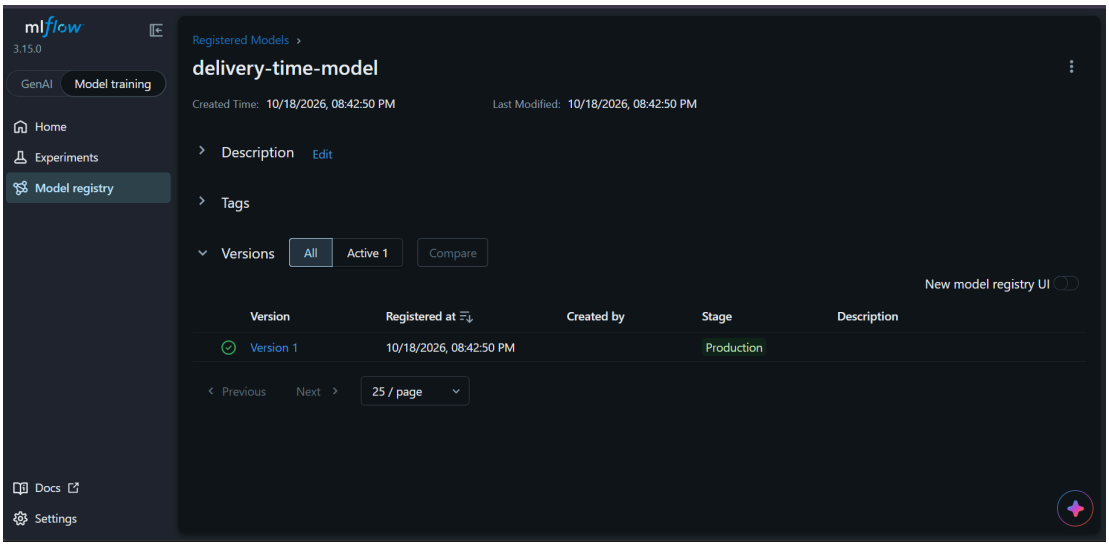
SCSE3040 Machine Learning Operations — Practical 07

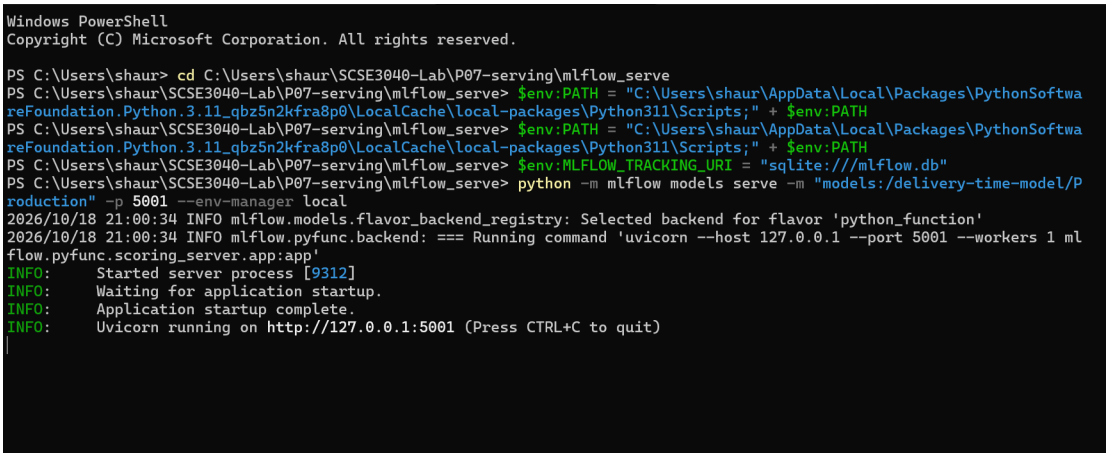
1. Student details

Name	Shaurya Nigam
Roll number	S24CSEU0497
Batch	89
Date of laboratory session	07/10/26

2. MLflow Model Registry — `delivery-time-model`, Version 1, Stage: Production



3. Terminal running `mlflow models serve` (port 5001)



4. Test request with curl and the prediction returned



Body sent (`request.json`): one order of 5 km, 20 min prep, traffic level 2, no rain.

```
{"dataframe_split": {"columns": ["distance_km", "prep_time_min", "traffic_level", "rain"], "data": [[5.0, 20, 2, 0]]}}
```

Prediction returned: **43.01 minutes**.

5. Model details

Registered model name	delivery-time-model
Version promoted	1
Run ID	32078c0c7b8f417cb0106e619e8d46fb
Port served on	5001

Practical 07 --- Putting the Model Behind an Address

A prediction service with FastAPI, input validation, and automatic docs

SCSE3040 Machine Learning Operations · Bennett University · Session 2026-27

Lectures this follows	L09

Course Outcome	CO4
Lab duration	120 minutes
Memory needed	about 450 MB (fine on a 4 GB or 8 GB machine)
Extra software	nothing beyond the course venv
Marks	10

Aim

1. Explain why a trained model is useless until something can call it.
2. Build a small web service with a /health and a /predict endpoint.
3. Let the framework reject bad input for you, before your code runs.
4. Test the whole service from Python, without starting a server.

What you need to know first

Your model currently lives inside a notebook on your laptop. A food delivery app cannot open your notebook. It needs to *ask* for a prediction and get an answer back, from anywhere, at any time.

The way software asks other software for things is an **API** (Application Programming Interface). For services on a network, the usual style is **REST**, and the pieces are simple:

- an **endpoint** is an address, such as `/predict`
- a **method** says what you want: `GET` to fetch something, `POST` to send something and get a result
- the body of the message is usually **JSON** (JavaScript Object Notation), a plain-text format of keys and values that looks like a Python dictionary
- the reply carries a **status code**: `200` means fine, `422` means your input was unacceptable, `500` means the service itself broke

FastAPI is the Python framework we use. Three things make it right for machine learning. It is fast. It checks incoming data for you and rejects bad requests before your code ever sees them. And it writes its own documentation page, which you get for free.

That middle point matters more than it sounds. A model given `traffic_level=9` when it was trained on 1 to 3 will not crash --- it will return a confident, wrong number. Rejecting impossible input at the door is the cheapest safety you will ever add.

Today you also learn to test a service **without running a server**. FastAPI gives you a `TestClient` that calls your app directly, in the same process. No ports, no browser,

nothing to leave running by mistake --- and it works identically inside an automated pipeline, which is where P11 will use it.

Before you start

- Practical P04 is finished. You can write a module and save a model.
- You know what a dictionary is. You do not need any web experience.

How to run a cell

Click on a grey code cell, then press **Shift + Enter**. The cell runs and the cursor moves to the next one. A number appears in the `[ ]` on the left when the cell has finished.

Run the cells in order, from the top. A later cell almost always uses something an earlier cell created. If you jump ahead you will see a `NameError`, which just means "you have not made that thing yet".

If everything goes wrong, use the menu: **Kernel -> Restart Kernel and Clear All Outputs**, then start again from the first cell. Nothing is damaged by doing this.

In [1]:
```
# Step 0 --- check the workbench before we start.
# This cell only looks; it changes nothing. Run it and read the last line.

import sys
from pathlib import Path

print("Python  :", sys.version.split()[0])
print("Folder  :", Path.cwd().name)

_missing = []
for _name in ['numpy', 'pandas', 'sklearn', 'fastapi', 'joblib']:
    try:
        __import__(_name)
    except ImportError:
        _missing.append(_name)

for _name in ['numpy', 'pandas', 'sklearn', 'fastapi', 'joblib']:
    _mark = "missing" if _name in _missing else "ok"
    print(f"  {_name:<14} {_mark}")

if _missing:
    print()
    print("STOP. Some libraries are missing:", ", ".join(_missing))
    print("Ask your instructor to run the setup in labs/SETUP.md.")
else:
    print()
    print("All good. You can carry on to Step 1.")
```

```
Python  : 3.11.9
Folder  : P07-serving
```

```
numpy          ok
pandas         ok
sklearn        ok
fastapi        ok
joblib         ok

All good. You can carry on to Step 1.
```

Step 0b --- the dataset

Every practical in this course uses the same 600 food deliveries. The next cell makes sure the file is there.

In [2]:
```python
# The delivery-time dataset every practical in this course uses.
# If the file is missing we build it again from the same seed, so every
# student in the room gets byte-for-byte the same 600 rows.

import csv
from pathlib import Path

import numpy as np

SEED = 42
N_ROWS = 600
DATA = Path("..") / "data" / "delivery_times.csv"


def make_delivery_csv(path=DATA):
    """Write the 600-row delivery dataset. Same formula as the lectures."""
    rng = np.random.default_rng(SEED)
    distance_km = np.round(rng.uniform(0.5, 12.0, N_ROWS), 2)
    prep_time_min = np.round(rng.uniform(5, 30, N_ROWS), 0)
    traffic_level = rng.integers(1, 4, N_ROWS)
    rain = rng.binomial(1, 0.25, N_ROWS)
    delivery_min = np.round(
        6.0
        + 3.1 * distance_km
        + 0.65 * prep_time_min
        + 4.2 * traffic_level
        + 5.5 * rain
        + rng.normal(0, 2.5, N_ROWS),
        1,
    )
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", newline="", encoding="utf-8") as fh:
        w = csv.writer(fh)
        w.writerow(["distance_km", "prep_time_min", "traffic_level",
                    "rain", "delivery_min"])
        for i in range(N_ROWS):
            w.writerow([distance_km[i], int(prep_time_min[i]),
                        int(traffic_level[i]), int(rain[i]), delivery_min[i]])
    return path


if not DATA.exists():
    make_delivery_csv()
```

```
    print("dataset rebuilt ->", DATA)
else:
    print("dataset found   ->", DATA)
```

```
dataset found   -> ..\data\delivery_times.csv
```

Walkthrough

Read each step, then run its cell.

Step 1 --- Train a model and save it

The service needs something to serve. Train once, save to a file, exactly as in P04.

In [3]:
```
from pathlib import Path

import joblib
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error
from sklearn.model_selection import train_test_split

WORK = Path("work")
WORK.mkdir(exist_ok=True)

FEATURES = ["distance_km", "prep_time_min", "traffic_level", "rain"]
orders = pd.read_csv(DATA)

X_train, X_test, y_train, y_test = train_test_split(
    orders[FEATURES], orders["delivery_min"],
    test_size=0.2, random_state=42,
)
model = LinearRegression().fit(X_train, y_train)
mae = mean_absolute_error(y_test, model.predict(X_test))

joblib.dump(model, WORK / "model.joblib")
print(f"model saved, MAE {mae:.2f} minutes")
```

```
model saved, MAE 1.92 minutes
```

Step 2 --- Describe what a valid order looks like

Before writing the service, decide what you will accept. This is a **schema**: the shape of the data and the rules each field must obey.

`pydantic` turns that description into real checking. Read the rules below in English: distance must be above 0 and at most 50; traffic must be 1, 2 or 3; rain must be 0 or 1.

In [4]:
```
from pydantic import BaseModel, Field

class Order(BaseModel):
    """One food order waiting to be delivered."""
```

```
    distance_km: float = Field(..., gt=0, le=50)
    prep_time_min: int = Field(..., ge=0, le=120)
    traffic_level: int = Field(..., ge=1, le=3)
    rain: int = Field(..., ge=0, le=1)


good = Order(distance_km=5.0, prep_time_min=20,
             traffic_level=2, rain=0)
print("accepted:", good)

try:
    Order(distance_km=5.0, prep_time_min=20,
          traffic_level=9, rain=0)
except Exception as problem:
    print()
    print("rejected, and here is why:")
    print(str(problem)[:300])
```

```
accepted: distance_km=5.0 prep_time_min=20 traffic_level=2 rain=0

rejected, and here is why:
1 validation error for Order
traffic_level
  Input should be less than or equal to 3 [type=less_than_equal, input_value=9, i
nput_type=int]
    For further information visit https://errors.pydantic.dev/2.13/v/less_than_eq
ual
```

You wrote no `if` statements. The rules live in the description of the data, which is where they belong.

Step 3 --- Write the service

Now the app itself, in its own file so that a server can run it later.

Read the three decorated functions. `@app.get("/health")` means "when someone does a GET on /health, run this". That is the whole idea of a web framework.

In [5]:
```
app_py = '''"The delivery-time prediction service."

from pathlib import Path

import joblib
import pandas as pd
from fastapi import FastAPI
from pydantic import BaseModel, Field

HERE = Path(__file__).resolve().parent
FEATURES = ["distance_km", "prep_time_min", "traffic_level", "rain"]

app = FastAPI(
    title="Delivery Time Predictor",
    description="Predicts how many minutes a food order will take.",
    version="1.0.0",
)

model = joblib.load(HERE / "model.joblib")
```

```
class Order(BaseModel):
    "One food order waiting to be delivered."

    distance_km: float = Field(..., gt=0, le=50, examples=[4.2])
    prep_time_min: int = Field(..., ge=0, le=120, examples=[12])
    traffic_level: int = Field(..., ge=1, le=3, examples=[2])
    rain: int = Field(..., ge=0, le=1, examples=[0])


class Prediction(BaseModel):
    delivery_min: float
    model_version: str


@app.get("/health")
def health():
    "Is the service up, and is a model loaded?"
    return {"status": "ok", "model_loaded": model is not None}


@app.post("/predict", response_model=Prediction)
def predict(order: Order):
    "Predict the delivery time for one order, in minutes."
    row = pd.DataFrame([order.model_dump()])[FEATURES]
    minutes = float(model.predict(row)[0])
    return Prediction(delivery_min=round(minutes, 1),
                      model_version=app.version)
'''

(WORK / "app.py").write_text(app_py, encoding="utf-8")
print("wrote", WORK / "app.py", f"({len(app_py)} characters)")
```

```
wrote work\app.py (1346 characters)
```

Step 4 --- Load the service without starting a server

`TestClient` runs your app inside this notebook. Nothing listens on a network port; nothing is left running when you close Jupyter.

You may see a `StarletteDeprecationWarning` about `httpx`. It is harmless noise from the library, not a problem with your code.

In [6]:
```
import importlib
import shutil
import sys

from fastapi.testclient import TestClient

if str(WORK.resolve()) not in sys.path:
    sys.path.insert(0, str(WORK.resolve()))


def reload_service():
    """Read app.py from disk again and give back a fresh client.

    The first line matters. Python keeps a compiled copy of every
    module and decides whether it is stale from the file's size and
```

```
    its timestamp, rounded to the nearest second. Change a file
    without changing its length, twice in one second, and Python can
    miss it entirely. Deleting the cache removes the guesswork.
    """
    shutil.rmtree(WORK / "__pycache__", ignore_errors=True)
    importlib.invalidate_caches()
    module = importlib.reload(importlib.import_module("app"))
    return module, TestClient(module.app)


app_module, client = reload_service()
print("service loaded:", app_module.app.title,
      "version", app_module.app.version)
```

```
service loaded: Delivery Time Predictor version 1.0.0
```

```
C:\Users\shaur\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2
kfra8p0\LocalCache\local-packages\Python311\site-packages\fastapi\testclient.py:
1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is depr
ecated; install `httpx2` instead.
  from starlette.testclient import TestClient as TestClient  # noqa
```

Step 5 --- Ask it whether it is alive

`/health` is the simplest endpoint and the most useful in production. Kubernetes will call it every few seconds in P09 to decide whether this copy of your service should receive traffic.

In [7]:
```
response = client.get("/health")

print("status code:", response.status_code)
print("body       :", response.json())
```

```
status code: 200
body       : {'status': 'ok', 'model_loaded': True}
```

`200` means fine. Note that a health check must be cheap --- do not make it run a prediction, or a slow model will look like an outage.

Step 6 --- Ask it for a prediction

A `POST` sends data. We send JSON describing one order and get JSON back.

In [8]:
```
order = {
    "distance_km": 5.0,
    "prep_time_min": 20,
    "traffic_level": 2,
    "rain": 0,
}

response = client.post("/predict", json=order)

print("status code:", response.status_code)
print("body       :", response.json())
print()
print("Predicted:", response.json()["delivery_min"], "minutes")
```

```
status code: 200
body       : {'delivery_min': 43.0, 'model_version': '1.0.0'}

Predicted: 43.0 minutes
```

That is your model, serving a request. Everything from here --- the container in P08, the cluster in P09 --- is about running *this* reliably, not about making it cleverer.

Step 7 --- Watch it refuse bad input

Now the safety. Send three impossible orders and read the status codes. Your `predict` function never runs for any of them.

In [9]:
```
bad_orders = {
    "traffic level 9 (only 1-3 exist)":
        {"distance_km": 5, "prep_time_min": 20,
         "traffic_level": 9, "rain": 0},
    "negative distance":
        {"distance_km": -3, "prep_time_min": 20,
         "traffic_level": 2, "rain": 0},
    "prep_time_min missing altogether":
        {"distance_km": 5, "traffic_level": 2, "rain": 0},
}

for description, payload in bad_orders.items():
    r = client.post("/predict", json=payload)
    print(f"{description:<36} -> {r.status_code}")

print()
print("Detail of the last rejection:")
print(r.json()["detail"][0]["msg"])
```

```
traffic level 9 (only 1-3 exist)     -> 422
negative distance                    -> 422
prep_time_min missing altogether     -> 422

Detail of the last rejection:
Field required
```

`422` means "I understood your request and it is not acceptable". The reply says which field was wrong and why, so the app calling you can show a sensible message to the user.

Without this, `traffic_level=9` would produce a confident prediction that is complete nonsense.

Step 8 --- The documentation you did not write

FastAPI publishes a description of itself at `/openapi.json`, and a browsable page built from it at `/docs`.

Nobody maintains this. It is generated from the code, so it cannot drift out of date.

In [10]:
```
spec = client.get("/openapi.json").json()
```

```
print("service :", spec["info"]["title"], spec["info"]["version"])
print("endpoints:")
for path, methods in spec["paths"].items():
    for method in methods:
        print(f"   {method.upper():<5} {path}")

print()
print("Fields /predict requires:")
schema = spec["components"]["schemas"]["Order"]
for field, rules in schema["properties"].items():
    limits = {k: v for k, v in rules.items()
              if k in ("minimum", "maximum", "exclusiveMinimum")}
    print(f"   {field:<16} {rules.get('type'):<8} {limits}")
```

```
service : Delivery Time Predictor 1.0.0
endpoints:
   GET   /health
   POST  /predict

Fields /predict requires:
   distance_km      number   {'maximum': 50.0, 'exclusiveMinimum': 0.0}
   prep_time_min    integer  {'maximum': 120.0, 'minimum': 0.0}
   traffic_level    integer  {'maximum': 3.0, 'minimum': 1.0}
   rain             integer  {'maximum': 1.0, 'minimum': 0.0}
```

Step 9 --- Run it for real, in a terminal

The `TestClient` is for testing. To let a phone app or a browser reach your service, you need a real server. **uvicorn** is that server.

We do not run it in the notebook: a server runs forever, so the cell would never finish. Open a terminal instead.

Press `Ctrl + C` in the terminal to stop it.

In [11]:
```
print("Run these two lines in a terminal:")
print()
print(f"  cd {(Path.cwd() / 'work')}")
print("  python -m uvicorn app:app --reload --port 8000")
print()
print("Then open http://127.0.0.1:8000/docs in your browser.")
print()
print("What you will see first: a page titled 'Delivery Time")
print("Predictor' with two rows, GET /health and POST /predict.")
print("Click POST /predict, then the 'Try it out' button, edit the")
print("example order, and press Execute. The answer appears below")
print("under 'Response body'.")
```

```
Run these two lines in a terminal:

  cd C:\Users\shaur\SCSE3040-Lab\P07-serving\work
  python -m uvicorn app:app --reload --port 8000

Then open http://127.0.0.1:8000/docs in your browser.

What you will see first: a page titled 'Delivery Time
Predictor' with two rows, GET /health and POST /predict.
Click POST /predict, then the 'Try it out' button, edit the
example order, and press Execute. The answer appears below
under 'Response body'.
```

Take a screenshot of `/docs` after a successful `Execute`, showing the response body. You need it for your submission.

Your turn

The walkthrough above is finished. Now you write some code.

There are **3 tasks**. Each one is small. Each one has a hint. Do them in order.

Where you see `# TODO`, replace that line with your own code. Do not delete the variable name on the left of the `=` sign --- the self-check at the bottom looks for exactly that name.

When you have tried all three, run the **self-check** cell at the end. It prints a table telling you which tasks are correct. You can run it as many times as you like.

Task T1 --- Add an endpoint that describes the model

```
Add a new endpoint `GET /model-info` to `work/app.py`. It must
return a
JSON object with exactly these three keys:

- `"features"` --- the list of four feature names, in order
- `"model_type"` --- the class name of the loaded model, as a
string
- `"version"` --- the app's version

Then reload the service and call it, storing the response body
in
`T1_info`.
```

> **Hint.** Append to the file with `open(WORK / "app.py", "a", encoding="utf-8")`. Copy the shape of the `/health` function. `type(model).__name__` gives `'LinearRegression'`. After writing the file, call `app_module, client = reload_service()` to pick up the change, then `client.get("/model-info").json()`.

In [12]:
```
extra_endpoint = '''


@app.get("/model-info")
def model_info():
    "Describe the model this service is running."
    return {
        "features": FEATURES,
        "model_type": type(model).__name__,
        "version": app.version,
    }
'''

with open(WORK / "app.py", "a", encoding="utf-8") as fh:
    fh.write(extra_endpoint)

# reload the service so it sees the new endpoint, then call it
app_module, client = reload_service()
T1_info = client.get("/model-info").json()

print(T1_info)
```

```
{'features': ['distance_km', 'prep_time_min', 'traffic_level', 'rain'], 'model_ty
pe': 'LinearRegression', 'version': '1.0.0'}
```

Task T2 --- Compare two orders through the service

Using `client.post`, ask the service for two predictions:

- a 3 km order, 15 minutes prep, traffic 1, **no** rain
- the same order **in the rain**

Store the two predicted minute values in `T2_dry` and `T2_wet`.

Use the service, not the `model` variable --- the point is to go through the API.

> **Hint.** Build a dictionary for each order and send it with `client.post("/predict", json=order)`. The number you want is `response.json()["delivery_min"]`.

In [13]:
```
dry_order = {"distance_km": 3.0, "prep_time_min": 15,
             "traffic_level": 1, "rain": 0}
wet_order = {"distance_km": 3.0, "prep_time_min": 15,
             "traffic_level": 1, "rain": 1}

# send each order to the service and read delivery_min from the reply
T2_dry = client.post("/predict", json=dry_order).json()["delivery_min"]
T2_wet = client.post("/predict", json=wet_order).json()["delivery_min"]

print("dry:", T2_dry, " wet:", T2_wet)
print("rain adds", round(T2_wet - T2_dry, 1), "minutes")
```

```
dry: 29.5  wet: 35.0
rain adds 5.5 minutes
```

Task T3 --- Refuse an order nobody could deliver

The service currently accepts a distance of 50 km. Our riders never go
beyond **25 km**.

Change the `Order` schema in `work/app.py` so that `distance_km` must be
25 or less, reload the service, and prove it:

- store the status code for a 30 km order in `T3_too_far`
- store the status code for a 10 km order in `T3_fine`

> **Hint.** Read the whole file with `read_text`, replace the text `le=50` with `le=25`, and write it back with `write_text`. Then call `app_module, client = reload_service()`. A rejected request gives `422`; an accepted one gives `200`.

In [14]:
```
source = (WORK / "app.py").read_text(encoding="utf-8")

# change the distance_km limit from 50 to 25 and save the file
source = source.replace("le=50", "le=25")
(WORK / "app.py").write_text(source, encoding="utf-8")

# reload the service so the new rule is used
app_module, client = reload_service()

# send a 30 km order and a 10 km order, keep their status codes
far_order = {"distance_km": 30, "prep_time_min": 15,
             "traffic_level": 1, "rain": 0}
near_order = {"distance_km": 10, "prep_time_min": 15,
              "traffic_level": 1, "rain": 0}

T3_too_far = client.post("/predict", json=far_order).status_code
T3_fine = client.post("/predict", json=near_order).status_code

print("30 km ->", T3_too_far, "   10 km ->", T3_fine)
```

```
30 km -> 422    10 km -> 200
```

Self-check

Run the cell below to mark your work.

In [15]:
```
# ----------------------------------------------------------------
# SELF-CHECK --- run this when you have attempted the tasks above.
# It never breaks your notebook. A task you have not done yet simply
# shows FAIL.
# ----------------------------------------------------------------

_results = []
```

```
def _check(label, fn):
    """Evaluate one graded condition without ever raising."""
    try:
        ok = bool(fn())
    except Exception:
        ok = False
    _results.append((label, ok))


_check('T1 | GET /model-info answers with 200', lambda: client.get('/model-info'
_check('T1 | it returns the three required keys', lambda: set(T1_info) == {'feat
_check('T1 | the values are correct', lambda: T1_info['features'] == ['distance_
_check('T2 | both are numbers in a plausible range', lambda: 20 < float(T2_dry)
_check('T2 | rain makes the same order take longer', lambda: float(T2_wet) > flo
_check('T2 | the values match what the service returns', lambda: abs(float(T2_dr
_check('T3 | a 30 km order is now rejected with 422', lambda: int(T3_too_far) ==
_check('T3 | a 10 km order is still accepted with 200', lambda: int(T3_fine) ==
_check('T3 | the new limit is really in the schema, not just in the test', lambd

print("======================================================================")
print("SELF-CHECK   Practical 07 --- Putting the Model Behind an Address")
print("======================================================================")
for _label, _ok in _results:
    print(f"  [{'PASS' if _ok else 'FAIL'}]  {_label}")
print("----------------------------------------------------------------------")
_passed = sum(1 for _, _ok in _results if _ok)
print(f"  {_passed} of {len(_results)} checks passed")
print("======================================================================")
if _passed == len(_results):
    print("Well done. Save the notebook and submit it.")
else:
    print("Read the FAIL lines above, fix those tasks, run this cell again.")
```

```
======================================================================
SELF-CHECK   Practical 07 --- Putting the Model Behind an Address
======================================================================
  [PASS]  T1 | GET /model-info answers with 200
  [PASS]  T1 | it returns the three required keys
  [PASS]  T1 | the values are correct
  [PASS]  T2 | both are numbers in a plausible range
  [PASS]  T2 | rain makes the same order take longer
  [PASS]  T2 | the values match what the service returns
  [PASS]  T3 | a 30 km order is now rejected with 422
  [PASS]  T3 | a 10 km order is still accepted with 200
  [PASS]  T3 | the new limit is really in the schema, not just in the test
----------------------------------------------------------------------
  9 of 9 checks passed
======================================================================
Well done. Save the notebook and submit it.
```

What to submit

1. This notebook, with every cell run and its output visible.
2. Your `work/app.py`.

3. A screenshot of `http://127.0.0.1:8000/docs` showing a successful prediction in the Response body panel.

Name your file `PXX_<your-roll-number>.ipynb` before you upload it.

How this practical is marked

What is marked	Marks
Walkthrough run end to end, service answers requests	3
Task T1 --- a new endpoint added	2
Task T2 --- predictions requested and compared	2
Task T3 --- validation tightened and proven	3
Total	**10**

Read more

- FastAPI --- First steps --- https://fastapi.tiangolo.com/tutorial/first-steps/
- FastAPI --- Request body and pydantic models --- https://fastapi.tiangolo.com/tutorial/body/
- FastAPI --- Testing with TestClient --- https://fastapi.tiangolo.com/tutorial/testing/
- Pydantic --- Field constraints --- https://docs.pydantic.dev/latest/concepts/fields/
- MDN --- HTTP response status codes --- https://developer.mozilla.org/en-US/docs/Web/HTTP/Status